Frontend ↔ Backend Integration — Lesson (Beginner)

Audience: Beginner web students who know basic HTML/CSS and some React and JavaScript.

Objectives

- Explain the architecture of this project (frontend + backend).
- Show how the frontend communicates with the backend (HTTP, JSON, auth token).
- Walk through concrete examples from this repository.
- Provide exercises and step-by-step demo instructions students can follow.

Project overview (this repo)

- Frontend: React + TypeScript app in `src/` (routes, pages, API helper).
  - Key file: src/api/api.ts
- Backend: Express + TypeScript server in `Backend/` with MongoDB.
  - Server entry: Backend/index.ts
  - Auth routes: Backend/routes/authRoutes.ts
  - Product routes: Backend/routes/productRoutes.ts
  - Auth controller: Backend/Controller/authController.ts
  - Product controller: Backend/Controller/productController.ts

High-level architecture

- Browser (React) → sends HTTP requests to API (JSON). Requests may include `Authorization: Bearer <token>` for protected endpoints.
- Backend (Express) exposes REST endpoints under `/api/*`, validates input, interacts with MongoDB via Mongoose models, and returns JSON responses.

How the frontend is configured to talk to the backend

- See src/api/api.ts: a shared `axios` instance with `baseURL` set to `http://localhost:4001/api` and a request interceptor that attaches a saved token from `localStorage` to every request.

Key endpoints and what they do

- POST /api/auth/signup → create a user account (see Backend/routes/authRoutes.ts).
  - Implementation: Backend/Controller/authController.ts#signUp
- POST /api/auth/login → log in, returns `{ token, user }` (see `login` in `authController.ts`).
- GET /api/products → list all products (public) — Backend/routes/productRoutes.ts → `getAllProducts`.
- POST /api/products → create a product (admin-only) → `createProduct` (see `productController.ts`).
- GET /api/products/:id → get single product by id.

Concrete frontend flow: Sign up & auto-login (example from this repo)

- Frontend uses `api.post('/auth/signup', formData)` and then `api.post('/auth/login', { email, password })` to receive a `token` and `user`.
- Example implementation: src/Pages/SignUp.tsx — the `handleSubmit` posts signup, then logs in, stores `token` and `user` in `localStorage`, and navigates home.

Authentication details (simple summary)

- Passwords are hashed with `bcrypt` before saving to the DB. See `authController.ts` (bcrypt usage).
- On successful login the server signs a JWT with `env.JWT_SECRET` and returns it. The frontend stores it in `localStorage` and the `api` client attaches it to future requests.
- Protected routes on the server use middleware `requireAuth` / `requireAdmin` (see `Backend/middleware/auth.ts`) to read the Authorization header, validate the token, and attach the user to the request.

Suggested lesson plan / class script (90 minutes)

1. 10m: High-level architecture and routes — open Backend/index.ts and src/api/api.ts.
2. 15m: Walk through signup/login flow — show `authRoutes` and `authController` then `src/Pages/SignUp.tsx`.
3. 15m: Live demo — run backend and frontend, sign up, open DevTools network to observe requests and responses.
4. 20m: Show product listing — `GET /api/products` from the frontend (create a simple fetch component or inspect network calls from existing product pages).
5. 15m: Add a protected route (admin) exercise — show product creation `POST /api/products` and middleware stack.
6. 15m: Q&A + exercises to try at home.

Class exercises (homework)

- Exercise 1: Add client-side validation to the signup form: show error messages before sending to the server.
- Exercise 2: Create a small `Profile` page that fetches the current user from `localStorage` and displays the name + role.
- Exercise 3: Implement logout: remove `token` and `user` from `localStorage` and redirect to login.
- Exercise 4 (stretch): Implement refresh token flow (explain why it's useful).

How to run this project locally (for demos)

1. Backend

- cd into `Backend` and install dependencies (if not already):

```
cd Backend
npm install
```

- Create a `.env` with the required environment variables (see `Backend/config/env.ts`), example:

```
PORT=4001
MONGO_URI=mongodb://localhost:27017/shoestore
JWT_SECRET=some_long_secret
```

- Start the backend server:

```
npm run dev
# or
node dist/index.js  # if compiled
```

2. Frontend

- From the project root:

```
npm install
npm run dev
```

- Open the frontend in the browser (Vite usually serves at http://localhost:5173) and use the Sign up page.

Turning this lesson into a PDF

- Option A (quick, reproducible): Convert this Markdown file to PDF using `pandoc` or your editor's Print→PDF.
- Option B (Node + Puppeteer): run the included `scripts/convert-to-pdf.js` script (requires `npm i puppeteer markdown-it`) to render the Markdown and export a `lesson.pdf`.

Where I put these files

- Lesson Markdown: `docs/Frontend-Backend-Integration-Lesson.md`
- Optional converter script: `scripts/convert-to-pdf.js`

Notes for instructors

- Emphasize separation of concerns: frontend handles UI and user interaction, backend enforces business rules, validation, and stores persistent data.
- Show how the same `token` saved in `localStorage` is the bridge that tells the server who the user is.
- Explain error codes (400 vs 401 vs 403 vs 404 vs 500) using the controllers in `Backend/Controller/*`.

References to open for students in the repo

- src/api/api.ts
- src/Pages/SignUp.tsx
- Backend/index.ts
- Backend/routes/authRoutes.ts
- Backend/routes/productRoutes.ts
- Backend/Controller/authController.ts
- Backend/Controller/productController.ts

-- End of lesson document --